

WOXSEN UNIVERSITY

B.Tech, Industry SDLC (PBL)

ASSIGNMENT REPORT

Assignment 38

Secure SDLC Policy

Name: Anshu

Roll Number: 24WU0101163

Module: Security

Submission type: PBL

October 2026

1. Objective

The aim of this assignment is to write a Secure SDLC policy, which is a set of rules that makes security part of every stage of building software instead of something added at the end. For each phase (requirements, design, coding, testing, deployment and maintenance) the policy gives a checklist of what the team must do and what proof they must keep. I also built a few small code demos to show that some of the key rules really work.

2. Tools Used

The policy itself is a document, so it was written in Microsoft Word. For the demos and diagrams I used the following:

- **Code:** Node.js 22 for the demo scripts, with the bcryptjs and pino packages, and the built-in node:sqlite module for a small test database.
- **Reference frameworks:** OWASP Top 10, OWASP ASVS and the STRIDE threat model.
- **Tools named in the policy:** SonarQube / Semgrep (SAST), OWASP ZAP (DAST), Dependency-Check / npm audit (SCA), Trivy (container scan), Gitleaks (secret scan), Jenkins Credentials (secrets).
- **Diagrams:** a Python script for the three diagrams.

3. Steps Followed

1. First I read about why security has to be part of every SDLC phase, and why a design mistake is much cheaper to fix than a breach.
2. Then I went through the course modules (secure coding, threat modeling, SAST/DAST/SCA, secret management, data privacy, incident response) and noted the security activities that came up.
3. Next I matched each activity to the phase where it fits best, so that no phase is left without a security control.
4. I wrote a checklist for each of the six phases, and for every item I also wrote what evidence the team should keep.
5. To make the policy easier to understand, I drew a phase diagram, a data flow diagram for an online payment feature, and a CI/CD pipeline with security gates.
6. Finally I wrote four small Node.js demos (SQL injection fix, password hashing, log redaction and a config check), ran each one and took screenshots of the output.

4. Policy Overview

4.1 Purpose and scope

This policy lists the minimum security work every project must complete in each phase of the SDLC. It applies to web, mobile and API projects, whether they are built in-house or by a vendor. The main idea is that security is built in, not bolted on.

4.2 Roles

Role	What they are responsible for
Product Owner	Sets security and compliance requirements, accepts leftover risk and signs off the release.

Role	What they are responsible for
Developers	Follow the coding standard, fix findings and write security tests.
QA / Testers	Design and run security test cases and block a release that fails the gate.
DevOps Engineer	Looks after secrets, pipeline security gates, hardened deployments and monitoring.
Security Champion	Leads threat modeling and design reviews and keeps the checklists up to date.

4.3 The six phases

Figure 1 shows the six phases and the main security activity in each. Maintenance feeds back into requirements, because problems found in production become the security requirements of the next release.

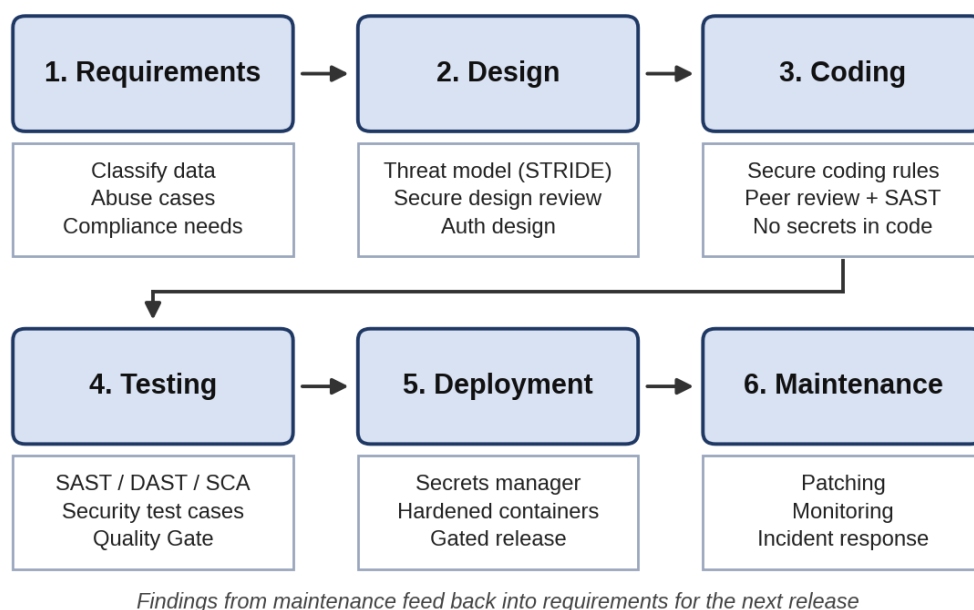


Figure 1: Secure SDLC phases and their main security activities

A phase counts as complete only when every item in its checklist is done, or has an approved exception (see section 6).

5. Phase-wise Security Checklists

5.1 Phase 1: Requirements

Before any design work, the team has to know what needs protecting and which rules apply.

#	Checklist item	Evidence
1	Identify and classify the data the system handles (public, internal, confidential, personal data such as Aadhaar, PAN and card numbers).	Data classification list
2	List the legal and compliance needs (DPDP Act 2023, GDPR, ISO 27001, PCI-DSS where card payments are involved).	Compliance list
3	Write security requirements next to the functional ones: login, roles, session timeout, audit logging, encryption.	Security section of the SRS

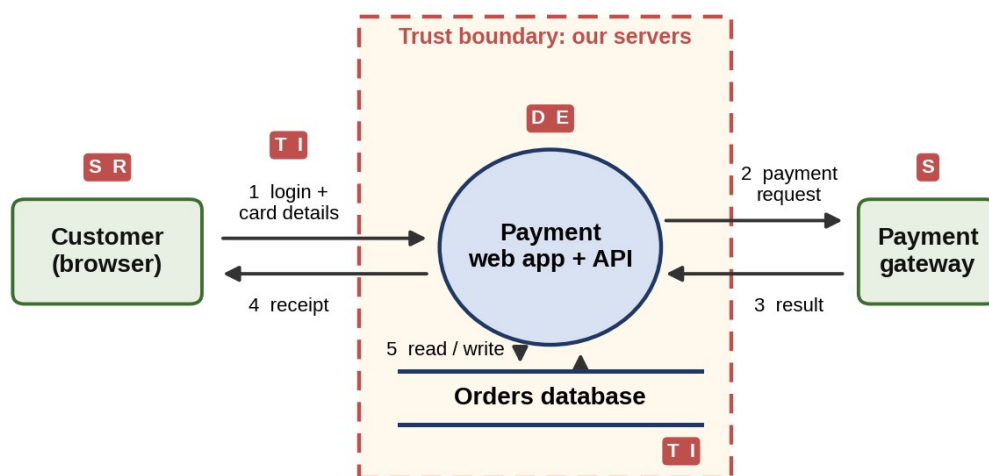
#	Checklist item	Evidence
4	Write abuse cases ("as an attacker I try to...") next to the user stories.	Abuse cases in the backlog
5	Add security acceptance criteria to every story that touches sensitive data.	Stories with Given/When/Then
6	Do an initial risk assessment and get the Product Owner to sign it off.	Signed risk register

5.2 Phase 2: Design

A flaw found at design time costs far less to fix than one found after a breach.

#	Checklist item	Evidence
1	Draw a data flow diagram with trust boundaries and run STRIDE threat modeling on it (see Figure 2).	DFD, threat table
2	Apply secure design principles: least privilege, defence in depth, secure by default, fail closed.	Design review notes
3	Design login and access control in one central place (SSO with OAuth 2.0 / OpenID Connect, MFA, role-based access).	Auth architecture diagram
4	Decide how data is protected in transit (TLS) and at rest (encryption, password hashing) and how keys are managed.	Encryption decision record
5	Pick approved, maintained frameworks and libraries only.	Approved components list
6	Hold a security design review before coding starts.	Review sign-off

As an example of threat modeling, Figure 2 is a data flow diagram for an online payment feature. The dashed box is the trust boundary: everything inside is our servers, and anything crossing it needs extra care. The red tags show which STRIDE threats apply at each point, and the table below lists how each one is handled.



Red tags = STRIDE threats: S Spoofing, T Tampering, R Repudiation, I Information disclosure, D Denial of service, E Elevation of privilege

Figure 2: Data flow diagram of an online payment feature with STRIDE threats

STRIDE threat	Where it applies	Mitigation
Spoofing	Customer login, gateway callback	Multi-factor authentication, signed gateway responses
Tampering	Amount in the request, orders database	Check the price on the server, parameterized queries
Repudiation	Customer denies making a payment	Audit log with user ID, time and IP address
Information disclosure	Card details, orders database, logs	TLS, encryption at rest, log redaction, never store the CVV
Denial of service	Login and payment endpoints	Rate limiting, timeouts, load balancing
Elevation of privilege	A normal user calling admin APIs	Role check on the server for every route

5.3 Phase 3: Coding

Developers follow one shared secure-coding standard, and tools check the code automatically.

#	Checklist item	Evidence
1	Follow a coding standard based on the OWASP Top 10 and OWASP ASVS.	Team coding standard
2	Validate all input on the server and use parameterized queries. Never build SQL by joining strings.	Code review checklist
3	Hash passwords with bcrypt or argon2. Never store them in plain text.	Auth code review
4	Never hard-code secrets, keys or tokens. Load them from environment variables or a secret store.	Gitleaks scan result
5	Show users generic error messages. Keep details in server logs and never leak stack traces.	Error handling review
6	Every change goes through a feature branch, a Pull Request, peer review and a protected-branch merge, with SAST (SonarQube or Semgrep) on each PR.	PR history, SAST report

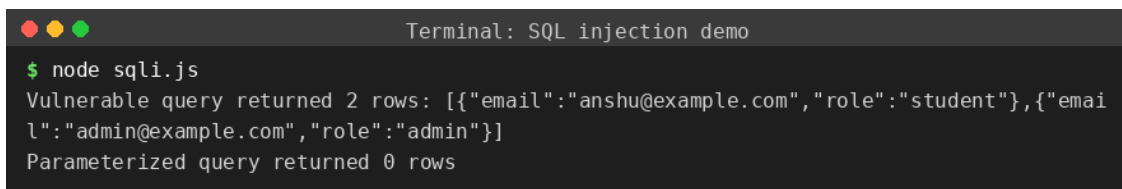
Example 1: SQL injection. If user input is joined into an SQL string, an attacker can change the meaning of the query. A parameterized query sends the input separately, so it is only ever treated as data.

```
const input = "x' OR '1'='1";    // attacker input

// BAD: input is joined into the SQL string
const bad = db.prepare(
  "SELECT email, role FROM users WHERE email = '" + input + "'"
).all();

// GOOD: placeholder, the driver treats input as data only
const good = db.prepare(
  "SELECT email, role FROM users WHERE email = ?"
).all(input);
```

When I ran this against a test table with two users, the vulnerable query returned both rows, including the admin account, while the parameterized query returned nothing (Figure 3).

A terminal window titled "Terminal: SQL injection demo" showing the output of a Node.js script. The script runs "node sqli.js" and displays the results of two database queries. The first query, labeled "Vulnerable query", returned 2 rows: one for a student user and one for an admin user. The second query, labeled "Parameterized query", returned 0 rows.

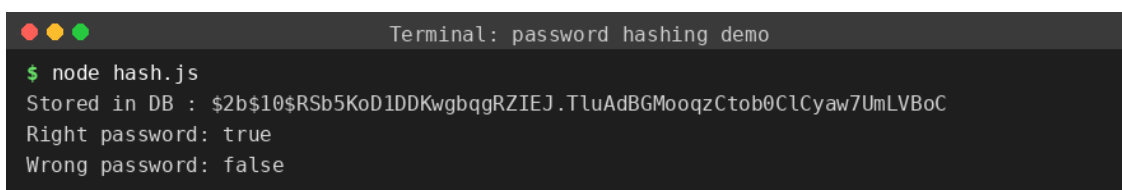
```
Terminal: SQL injection demo
$ node sqli.js
Vulnerable query returned 2 rows: [{"email":"anshu@example.com","role":"student"}, {"email":"admin@example.com","role":"admin"}]
Parameterized query returned 0 rows
```

Figure 3: Output of the SQL injection demo

Example 2: password hashing. Passwords must never be stored as plain text. bcrypt turns the password into a salted hash, and at login the app compares the attempt against that hash.

```
const bcrypt = require('bcryptjs');

const hash = await bcrypt.hash(password, 10);    // store this, not the password
const ok = await bcrypt.compare(attempt, hash);  // true or false at login
```

A terminal window titled "Terminal: password hashing demo" showing the output of a Node.js script. The script runs "node hash.js" and displays the results of password hashing and comparison. It shows a password being hashed into a long string, and then compares it against the stored hash, returning "true" for a right password and "false" for a wrong password.

```
Terminal: password hashing demo
$ node hash.js
Stored in DB : $2b$10$RSb5KoD1DDKwgbqgRZIEJ.TluAdBGMooqzCtob0ClCyaw7UmLVBoC
Right password: true
Wrong password: false
```

Figure 4: Output of the password hashing demo

5.4 Phase 4: Testing

The team has to prove the controls work and catch vulnerabilities before release.

#	Checklist item	Evidence
1	Run SAST (code), SCA (dependencies), DAST (running app, for example OWASP ZAP) and container scanning (Trivy) in the pipeline.	Scan reports
2	Write security test cases: negative tests, broken access control, expired or tampered tokens, file upload abuse, injection and XSS attempts.	Security test cases

#	Checklist item	Evidence
3	Test only in non-production environments with synthetic or anonymized data, never real customer data.	Test data policy
4	Set a Quality Gate: no open Critical or High finding may go to release.	Quality Gate result
5	Add a regression test for every security bug that gets fixed.	Regression suite
6	Run a penetration test or security review before a major release.	Pen-test report

Most of these checks can run automatically on every commit. Figure 5 shows a CI/CD pipeline where the orange stages are security gates. If any gate fails, the pipeline stops and the build never reaches production.

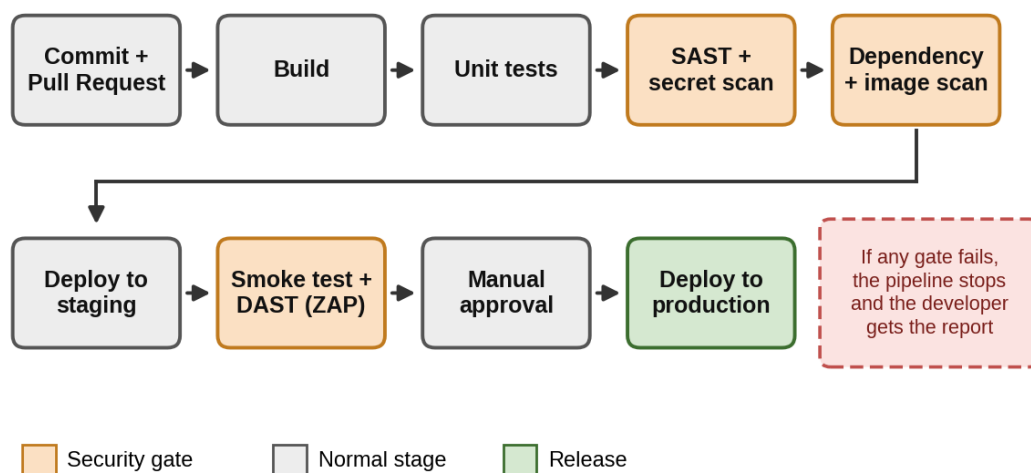


Figure 5: CI/CD pipeline with security gates

The same idea written as a Jenkinsfile is shown below. This is a sample for the policy and I did not run it on a Jenkins server.

```

pipeline {
  agent any
  stages {
    stage('Build') { steps { sh 'npm ci && npm run build' } }
    stage('Unit tests') { steps { sh 'npm test' } }
    stage('Secret scan') { steps { sh 'gitLeaks detect --source .' } }
    stage('SAST') { steps { sh 'semgrep --config auto --error' } }
    stage('Dependency scan') { steps { sh 'npm audit --audit-level=high' } }
    stage('Image scan') { steps {
      sh 'trivy image --exit-code 1 --severity HIGH,CRITICAL myapp:latest' } }
    stage('Deploy staging') {
      environment { DB_PASSWORD = credentials('db-password') } // masked in logs
      steps { sh 'docker compose up -d' }
    }
  }
}

```

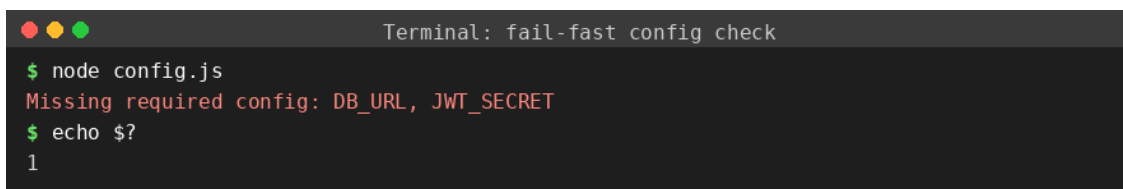
5.5 Phase 5: Deployment

Release safely, keep secrets protected and have a way back if something goes wrong.

#	Checklist item	Evidence
1	Keep secrets in a secret manager (Jenkins Credentials, GitHub Secrets, Vault) and check that they show as **** in build logs.	Masked build log
2	Keep configuration in environment variables, separate for dev, staging and production. The app must refuse to start if config is missing.	Config schema
3	Use minimal, scanned container images and run containers as a non-root user.	Dockerfile, Trivy report
4	Turn on HTTPS, security headers (CSP, HSTS), rate limiting and log redaction for passwords, tokens and card numbers.	Header and log samples
5	Let the pipeline block the deploy if a security scan or Quality Gate fails.	Failed gate screenshot
6	Raise a change request, get it approved and prepare a rollback plan before go-live.	Change ticket, rollback plan

Example 3: fail-fast configuration. The app reads secrets from environment variables, and it refuses to start if any of them is missing, so a bad deployment is caught immediately instead of failing later.

```
const required = ['DB_URL', 'JWT_SECRET'];
const missing = required.filter(k => !process.env[k]);
if (missing.length) {
  console.error('Missing required config: ' + missing.join(', '));
  process.exit(1);
}
```

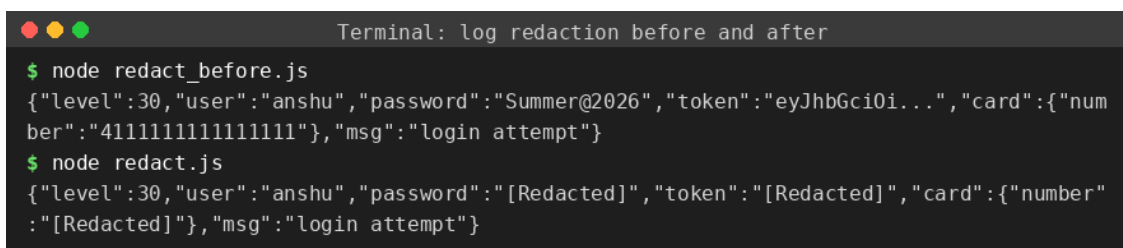


```
Terminal: fail-fast config check
$ node config.js
Missing required config: DB_URL, JWT_SECRET
$ echo $?
1
```

Figure 6: The app stops with exit code 1 when config is missing

Example 4: log redaction. Logs are often copied to other tools, so passwords, tokens and card numbers must never appear in them. The pino logger can hide chosen fields.

```
const pino = require('pino');
const log = pino({ redact: ['password', 'token', 'card.number'] });
```



```
Terminal: log redaction before and after
$ node redact_before.js
{"level":30,"user":"anshu","password":"Summer@2026","token":"eyJhbGciOi...", "card":{"number":"4111111111111111"},"msg":"login attempt"}
$ node redact.js
{"level":30,"user":"anshu","password":"[Redacted]","token":"[Redacted]","card":{"number":"[Redacted]"},"msg":"login attempt"}
```

Figure 7: Log line before and after redaction

5.6 Phase 6: Maintenance

New vulnerabilities keep appearing after launch, so the work does not stop at release.

#	Checklist item	Evidence
1	Patch on a fixed timeline: Critical within 7 days, High within 30 days, Medium within 90 days.	Patch log
2	Watch dependencies continuously (Dependabot, Dependency-Check) and rescan images regularly.	Alert history
3	Monitor logs and metrics for attacks and outages, with alert thresholds and an on-call process.	Dashboards, alert rules
4	Follow an incident response plan: detect, contain, eradicate, recover, then write a blameless postmortem.	IR plan, postmortems
5	Review user and admin access every quarter and remove leavers immediately.	Access review record
6	Test backups by restoring them. Retire old systems and delete data that is no longer needed.	Restore test, decommission record

6. Governance and Exceptions

- The security gates in the CI/CD pipeline are mandatory. A failed gate blocks the release.
- An exception needs a written reason, a named owner, extra controls to cover the gap and an expiry date, approved by the Product Owner and the Security Champion.
- The policy is reviewed once a year and after every serious incident.
- Evidence for every checklist item is kept with the project so it can be shown in an audit.

7. Results

The final policy has six checklists with 36 items in total, plus four working demos. The table compares what happens without and with the controls, based on the demo runs.

Check	Before (insecure way)	After (policy followed)
SQL injection input	String-joined query returned 2 rows, including the admin account	Parameterized query returned 0 rows
Password storage	Plain text password would be stored	bcrypt hash stored, wrong password rejected
Passwords, tokens and card numbers in logs	All three visible in the log line	All three shown as [Redacted]
App started with missing config	Nothing stops it, errors appear later	Exits with code 1 and a clear message

Code repository: [PASTE YOUR GITHUB REPOSITORY LINK HERE]

8. Observations and Conclusion

Observations

- The same rule often shows up in more than one phase. Secrets, for example, are banned in code, then scanned for in testing, then stored safely in deployment.
- The SQL injection demo surprised me the most. The only difference between the two versions is how the query is written, but one leaks the whole table.
- Most checks can be automated in the pipeline. That is what turns a policy from advice into something that is actually enforced.
- The requirements and design checklists are the cheapest to follow but easy to skip. Security also does not end at release, because patching and incident response are where many real breaches happen.

Conclusion

A Secure SDLC policy makes security a routine with clear owners and evidence at each phase. Following these checklists catches problems early, keeps fixes cheap, supports compliance with the DPDP Act, GDPR and ISO 27001, and lowers the risk of a breach after go-live. The demos showed that the key rules are simple to apply in code.

9. References

- OWASP Top 10 and OWASP Application Security Verification Standard (ASVS), owasp.org
- OWASP Threat Dragon and the STRIDE threat model
- Digital Personal Data Protection Act, 2023 (India) and the General Data Protection Regulation (GDPR)
- Node.js, bcryptjs and pino documentation
- Course material: Industry SDLC assignments, Security and Data Privacy modules